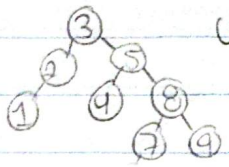
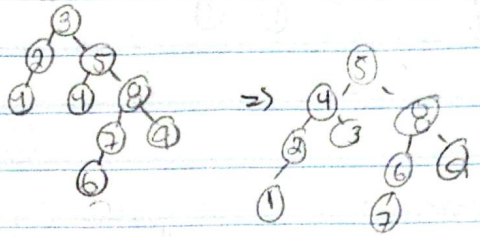
Q1 a) The AVL Tree is

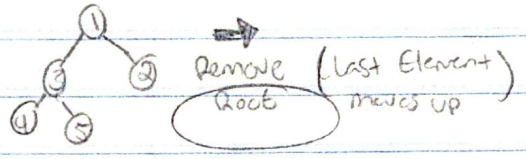
b)  Unbalanced

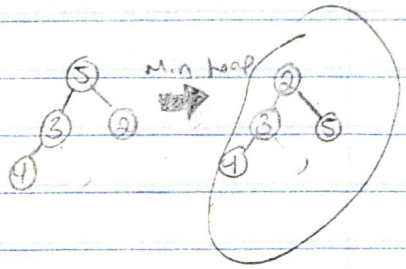
Add 6:



Q2 a)

you can only Remove Root



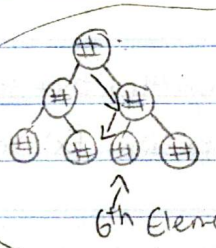


b) path to 6th Element of heap

6 ÷ 2 = 3 (0)

3 ÷ 2 = 1.5 (1) => Binary = 110

1 ÷ 2 = 0.5 (1)



, The path to the sixth element of a heap is to take its binary representation of 6 (110) and go from there. The first element is dropped for the head, And then we follow that pattern, (0 = left, 1 = Right). So we have 10 left, so we go Right and then left to find 6th element.

c) 2026th element 0101111111

2026 ÷ 2 = 1013 (0)	31 ÷ 2 = 15.5 (1)
1013 ÷ 2 = 506.5 (1)	15 ÷ 2 = 7.5 (1)
506 ÷ 2 = 253 (0)	7 ÷ 2 = 3.5 (1)
253 ÷ 2 = 126.5 (1)	3 ÷ 2 = 1.5 (1)
126 ÷ 2 = 63 (0)	1 ÷ 2 = 0.5 (1)
63 ÷ 2 = 31.5 (1)	

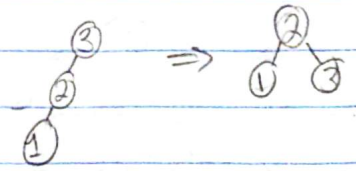
Binary = 11111101010, To find the 2026th element is Similar to the 6th, we will follow the pattern. Drop the first number for the head and then follow the pattern. Here's the pattern: Right (x5), Left, Right, Left, Right, Left

this is the path to the 2026th Element

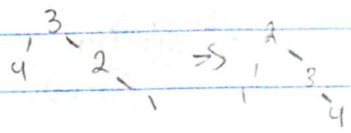
CamScanner

Q3 4 Rotations:

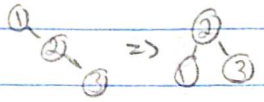
LL Rotation



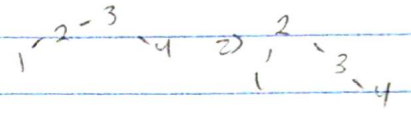
LR Rotation



RR Rotation



RL Rotation:



Q4 Linear Regression: A Supervised Machine Learning Algorithm that takes labeled data and splits it evenly across a straight line

Logistic Regression: A Supervised machine learning Algorithm that takes labeled data, creates a threshold, and does Binary (0,1) categorization. Ex: Yes/No

Supervised Machine Learning: Supervised Machine Learning is the type of machine learning that uses labeled data to find an expected outcome usually to train a model.

unsupervised machine learning: A type of machine learning that uses unlabeled data to find hidden patterns.

CamScanner

Q5 This code would

#1 : Somewhere

#2 : Everything

#3 : Anything

Q6 The code prints

⇒ Let's EAt, Grandma!

① LEt's EAt, GrAndmA! Let's EAt, GrAndmA

② Let's eat, Grandma!

CamScanner

Q7 #1: [1, 7, 4, 2, 6, 5, 6, 2, 4, 7]

#2: [2, 2, 4, 4]

#3: [2, 2, 2, 2, 2, 2, 2]

#4: [2, 2, 2]

#5: [2, 2, 2, 2, 2, 2, 2]

CamScanner

```
Class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

8 Class LinkedList:
    def __init__(self):
        self.head = None
    def insert(self, value):
        new = Node(value
        if self.head is None:
            self.head = new
            return
        current = self.head
        while current is not None:
            current = current.next
            return True
        current.next = new
        return False
    def show(self):
        current = self.head
        while current is not None:
            print(current.value, end = " →")
        current = current.next
        print("None")
```